



**POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
COLLEGE OF COMPUTER AND INFORMATION SCIENCES
DEPARTMENT OF COMPUTER SCIENCE**

Catch Me If You Can

A Game Development Proposal

Presented to the Department of Computer Science

Polytechnic University of the Philippines

Sta. Mesa, Manila

In Partial Fulfillment of the Requirements for the Course

COSC 304: Introduction to Artificial Intelligence

by

**Lasam, Vince Michael
Mercado, Jeff Petterson
Nicolas, John Rich
Paredes, Lian Luigi**

**BSCS 3-1
Group 6**

Asoc. Prof. Ria A. Sagum
April 2026

I. Introduction

Catch Me If You Can is a 2D top-down strategic defense game developed for PC (Windows/Mac) where the player takes on the role of the human immune system, fighting back against an increasingly intelligent and adaptive AI-controlled virus. The game is designed for all ages and presents its biological concepts in an accessible, visually engaging way without sacrificing scientific grounding.

The game draws its initial inspiration from Plague Inc., the popular mobile strategy game where the player controls a pathogen and guides it toward wiping out humanity. Catch Me If You Can flips this premise entirely — instead of being the disease, you are the cure. Where Plague Inc. rewards the player for outsmarting a passive human response system, Catch Me If You Can challenges the player to outsmart an AI virus that actively evolves and mutates in direct response to every decision made. The enemy is not scripted or static; it watches, learns, and adapts.

Another key distinction is the energy expenditure system. In Plague Inc., resources are abstract and consequence-free in terms of biological realism. In Catch Me If You Can, every immune action the player deploys — from sending white blood cells to triggering a fever — costs Energy Points (EP), a resource grounded in actual immunological science. The human immune system is metabolically expensive to operate; fever alone increases the body's metabolic rate by approximately 10–13% per degree Celsius of temperature rise, and the proliferation of white blood cells during an immune response draws heavily from the body's glucose and protein reserves. This system forces the player to think strategically about resource management rather than spamming the most powerful response available, mirroring how a real immune system must balance aggression with sustainability.

II. Game Proper

a. Objective of the Game

The player's goal is to eliminate the virus entirely before the host's Severity Index reaches 100%. The Severity Index is a percentage-based meter that tracks how critically the virus has compromised the host. It rises as the virus spreads through organs, mutates past defenses, and inflicts systemic damage. At 100% severity, the host is considered beyond recovery and the game ends in a loss.

Victory is achieved when all active virus cells are eradicated and the Severity Index is pushed back below a critical threshold, representing a full recovery. Critically, the **Infection Rate** and **Severity Index** are two distinct meters: the infection rate measures how many virus cells remain active — bringing it to zero wins the game regardless of remaining severity. The severity index

measures how lethal the virus has been to the host; a high severity index with zero infection rate still counts as a player victory.

b. Rules of the Game

The Playing Field

The game takes place on a top-down 2D map of the human body divided into interconnected organ zones: the **Lungs** (primary entry point), the **Bloodstream** (the virus's highway), the **Lymph Nodes** (the player's main resource generator), the **Gut**, the **Heart**, and the **Brain** (the critical final zone). Each zone has its own infection rate modifier and defense capacity. Healthy zones generate EP passively each round. Infected zones generate reduced EP, creating natural urgency — the longer the player waits to defend a zone, the fewer resources they have to work with.

Energy Points (EP) System

All player actions consume EP, which regenerates each round based on the number of healthy organ zones remaining. The costs are designed around real immunological metabolic demands:

Action	EP Cost	Effect
Deploy White Blood Cells	10 EP	Patrol a zone; reduce virus spread rate
Release Antibodies	25 EP	Target a specific strain; neutralize more effectively than WBCs
Trigger Inflammation	20 EP	Quarantine a zone; slight host HP damage per round
Activate Fever Response	35 EP	Global virus slowdown for two rounds; not consecutive
Deploy Memory Cells	15 EP	Counter a previously seen mutation at double speed
Cytokine Burst	50 EP	Massive damage to all infected zones; +10% Severity Index

Table 1. Energy Points (EP) system and immune actions.

Core Rules

The virus begins in the Lungs each game and spreads using a Breadth-First Search pattern, prioritizing zones with the fewest active defenses. At the end of each round, the AI evaluates which of the player's defenses dealt the most damage using a Weighted Response System and selects a counter-mutation from its Decision Tree accordingly. Mutations stack over time, making the virus progressively harder to counter with the same strategies.

The Severity Index rises under the following conditions:

- The virus occupies a new organ zone (+5%)
- A mutation goes uncontested for a full round (+3% per round)
- The Cytokine Burst is used (+10%)
- The Brain zone becomes infected (+25% immediately)

It decreases when virus cells are eliminated, zones are cleared, and the player sustains a clean round with no new infections.

Difficulty Modes

Mode	Turn Style	Mutation Rate	EP Regen	Entry Points
Casual	Turn-based	Every 2 rounds (telegraphed)	Generous	Lungs only
Epidemic	Turn-based	Every round (no warning)	Standard	Lungs only
Pandemic	Hybrid real-time + turn-based	Every round (no warning)	Reduced	3 random zones simultaneously

Table 2. Difficulty mode comparison.

III. Implementations of AI Algorithm

a. AI Algorithms Implemented in the Game

The game employs four complementary AI algorithms, each governing a distinct layer of the virus's behavior — from long-term strategic adaptation down to immediate tactical mutation selection. Together, they create an adversary that feels genuinely intelligent and unpredictable across all difficulty levels.

3.1 Breadth-First Search (BFS) — Virus Propagation

Breadth-First Search was selected to govern the virus's spatial propagation across the anatomical map. Rather than spreading randomly or along a fixed path, the virus needs to make structurally sound decisions about which organ zone to infect next. BFS is ideal because it explores all immediately adjacent zones before moving deeper into the body, naturally modeling how a real pathogen spreads through proximity and contact before reaching more distant organs.

In game terms, the anatomical map is treated as an undirected graph where each organ zone is a node and each physiological connection is an edge. BFS begins from the currently infected zone(s) and expands outward layer by layer. Critically, the traversal is weighted by each zone's active defense count — always prioritizing the neighbor with the fewest active defenses. This means the virus opportunistically flows toward whatever path of least resistance the player has left open, rewarding attentive players and punishing those who concentrate defenses in one area.

BFS also integrates naturally with Pandemic mode, where multiple simultaneous infection entry points exist. Each starting zone spawns its own independent BFS frontier, allowing multiple virus strains to expand concurrently without interfering with each other's traversal logic.

Pseudocode:

```
FUNCTION SpreadVirus(InfectedZones, OrganGraph):  
    // Initialize the BFS frontier with all currently infected zones  
    QUEUE = InitializeQueue(InfectedZones)  
    VISITED = SET(InfectedZones)  
  
    WHILE QUEUE is not empty:  
        CURRENT_ZONE = Dequeue(QUEUE)
```

```

// Get all organ zones directly connected to the current zone
NEIGHBORS = GetAdjacentZones(CURRENT_ZONE, OrganGraph)

// Filter to only zones not yet infected or visited this pass
UNVISITED_NEIGHBORS = FILTER NEIGHBORS where zone NOT IN VISITED

// Sort by fewest active defenses — virus targets weakest zones first
SORTED_NEIGHBORS = SortByDefenseCount(UNVISITED_NEIGHBORS, ASCENDING)

FOR EACH zone IN SORTED_NEIGHBORS:
    IF zone.defenseCount < SPREAD_RESISTANCE_THRESHOLD:
        InfectZone(zone)
        IncreaseSeverityIndex(+5%)
        Enqueue(Queue, zone)
    VISITED.Add(zone)

RETURN UpdatedInfectedZones
END FUNCTION

```

3.2 Minimax with Alpha-Beta Pruning — Strategic Decision-Making

Minimax with Alpha-Beta Pruning governs the virus's forward-looking strategic decision-making — specifically, its ability to evaluate not just the immediate outcome of a move, but the best possible outcome several turns ahead, assuming the player will always respond optimally.

The algorithm models the virus as the **maximizing player** (trying to push the Severity Index as high as possible) and the immune system as the **minimizing player** (trying to suppress it). At the start of each virus turn, Minimax constructs a game tree projecting several rounds into the future, evaluating all possible virus actions and all likely player counter-responses at each branch. The virus selects the action that leads to the highest guaranteed Severity Index gain even under the assumption that the player makes the best possible move at every step.

Alpha-Beta Pruning eliminates branches of the game tree that cannot possibly influence the final decision, significantly reducing the number of nodes evaluated without changing the outcome. This is critical for real-time viability in Pandemic mode. Tree depth scales with difficulty: Casual = 2, Epidemic = 4, Pandemic = 6.

Pseudocode:

```
FUNCTION MinimaxVirusTurn(GameState, Depth, Alpha, Beta, IsVirusTurn):
```

```
    // Base case: maximum lookahead depth reached or game is over
```

```
    IF Depth == 0 OR IsTerminalState(GameState):
```

```
        RETURN EvaluateBoardState(GameState)
```

```
    // Heuristic: weighted sum of SeverityIndex, InfectedZones, PlayerEP
```

```
    IF IsVirusTurn: // Virus is the maximizing player
```

```
        MaxScore = -INFINITY
```

```
        FOR EACH action IN GetVirusActions(GameState):
```

```
            NewState = SimulateVirusAction(GameState, action)
```

```
            Score = MinimaxVirusTurn(NewState, Depth - 1, Alpha, Beta, FALSE)
```

```
            MaxScore = MAX(MaxScore, Score)
```

```
        // Alpha-Beta Pruning
```

```
        Alpha = MAX(Alpha, Score)
```

```
        IF Beta <= Alpha:
```

```
            BREAK // Prune remaining branches
```

```
    RETURN MaxScore
```

```
ELSE: // Immune system is the minimizing player
```

```
    MinScore = +INFINITY
```

```

FOR EACH action IN GetPlayerActions(GameState):
    NewState = SimulatePlayerAction(GameState, action)
    Score = MinimaxVirusTurn(NewState, Depth - 1, Alpha, Beta, TRUE)
    MinScore = MIN(MinScore, Score)

    // Alpha-Beta Pruning
    Beta = MIN(Beta, Score)
    IF Beta <= Alpha:
        BREAK // Prune remaining branches

RETURN MinScore
END FUNCTION

FUNCTION SelectBestVirusMove(GameState):
    BestAction = NULL
    BestScore = -INFINITY
    Alpha = -INFINITY
    Beta = +INFINITY

    // Difficulty scales tree depth: Casual = 2, Epidemic = 4, Pandemic = 6
    SearchDepth = GetDifficultyDepth()

    FOR EACH action IN GetVirusActions(GameState):
        NewState = SimulateVirusAction(GameState, action)
        Score = MinimaxVirusTurn(NewState, SearchDepth, Alpha, Beta, FALSE)

        IF Score > BestScore:

```

```
BestScore = Score

BestAction = action

RETURN BestAction

END FUNCTION
```

3.3 Decision Tree — Mutation Selection

Decision Tree Learning governs the virus's immediate tactical responses, specifically its mutation selection process. At the end of each round, the Weighted Response System (WRS) scores each of the player's deployed defenses by how much damage they dealt. The defense with the highest weighted score is flagged as the player's dominant strategy, and this result is fed into the Decision Tree, which traverses a hierarchy of conditions to select the appropriate counter-mutation.

Decision trees are computationally lightweight, making them ideal for evaluating immediate conditions without significant processing overhead. They also provide a clear structure for difficulty scaling — deeper decision trees in Epidemic mode allow the virus to select compound mutations that counter two defense types simultaneously.

Pseudocode:

```
FUNCTION SelectCounterMutation(PlayerDefenses, VirusStrains):
    // Determine the player's primary strategy this round
    HEAVIEST_DEFENSE = EvaluateWeightedResponse(PlayerDefenses)

    // Traverse the decision tree based on the active defense
    IF HEAVIEST_DEFENSE is "WhiteBloodCells":
        RETURN "Evade_Phagocytosis_Mutation"

    ELSE IF HEAVIEST_DEFENSE is "Antibodies":
        IF VirusStrains.hasMutation("Antigenic_Drift"):
            RETURN "Antigenic_Shift_Mutation" // Compound mutation
```

```

ELSE:
    RETURN "Antigenic_Drift_Mutation"

ELSE IF HEAVIEST_DEFENSE is "Inflammation":
    RETURN "Thermal_Resistance_Mutation"

ELSE IF HEAVIEST_DEFENSE is "FeverResponse":
    RETURN "Heat_Shock_Proteins_Mutation"

ELSE:
    // Default behavior if player defenses are low
    RETURN "Accelerated_Replication_Mutation"
END FUNCTION

```

3.4 Reinforcement Learning (Q-Learning) — Strategic Adaptation

While the Decision Tree handles immediate tactical counters, Reinforcement Learning (RL) governs the virus's overarching strategic adaptation across rounds. The game is framed as a biological arms race where the enemy 'watches, learns, and adapts.' RL is suited for this dynamic adversary role because it allows the AI to optimize its long-term strategy by maximizing a reward signal — in this case, pushing the host's Severity Index toward the 100% failure threshold.

Through a Q-Learning framework, the virus evaluates the current state of the anatomical map (healthy vs. infected zones, player EP reserves) and takes actions (prioritizing specific organ zones via BFS) to maximize its reward (Severity Index gains). The Q-table is updated at the end of every round using the Bellman equation, with the reward signal reflecting the full outcome of both the virus and immune system turns. This ensures the virus becomes genuinely unpredictable and increasingly challenging as runs progress.

Pseudocode:

```

INITIALIZE Q_Table[States, Actions] to 0
SET LearningRate (alpha), DiscountFactor (gamma), ExplorationRate (epsilon)

```

```

FUNCTION RunVirusRL_Turn(CurrentBoardState):
    // Define the current state based on game variables
    CURRENT_STATE = GetGameState(InfectedZones, PlayerEP, SeverityIndex)

    // Epsilon-Greedy action selection for exploration vs. exploitation
    IF RandomNumber() < ExplorationRate:
        ACTION = SelectRandomAction(AvailableSpreadTargets, AvailableMutations)
    ELSE:
        ACTION = GetActionWithHighestQValue(CURRENT_STATE, Q_Table)

    // The virus executes its move
    EXECUTE ACTION

    // Wait for the round to conclude and assess the damage
    WAIT FOR Player_Turn_End

    // Observe the new environment state and calculate the reward
    NEW_STATE = GetGameState(InfectedZones, PlayerEP, SeverityIndex)
    REWARD = CalculateSeverityIndexIncrease(CURRENT_STATE, NEW_STATE)
    // High reward if Severity Index spiked, penalty if it dropped

    // Bellman Equation to update the Q-Table
    BEST_FUTURE_Q = GetMaxQValue(NEW_STATE, Q_Table)
    Q_Table[CURRENT_STATE, ACTION] = Q_Table[CURRENT_STATE, ACTION] +
        alpha * (REWARD + gamma * BEST_FUTURE_Q - Q_Table[CURRENT_STATE,
ACTION])

END FUNCTION

```

b. Software Architecture

The software architecture of Catch Me If You Can describes the complete game loop as a structured sequence of phases executed each round. The architecture is designed around a core design principle: the virus always acts first, because the immune system has no biological trigger without an existing threat. The diagram below (Figure 1) illustrates the full round-by-round flow.

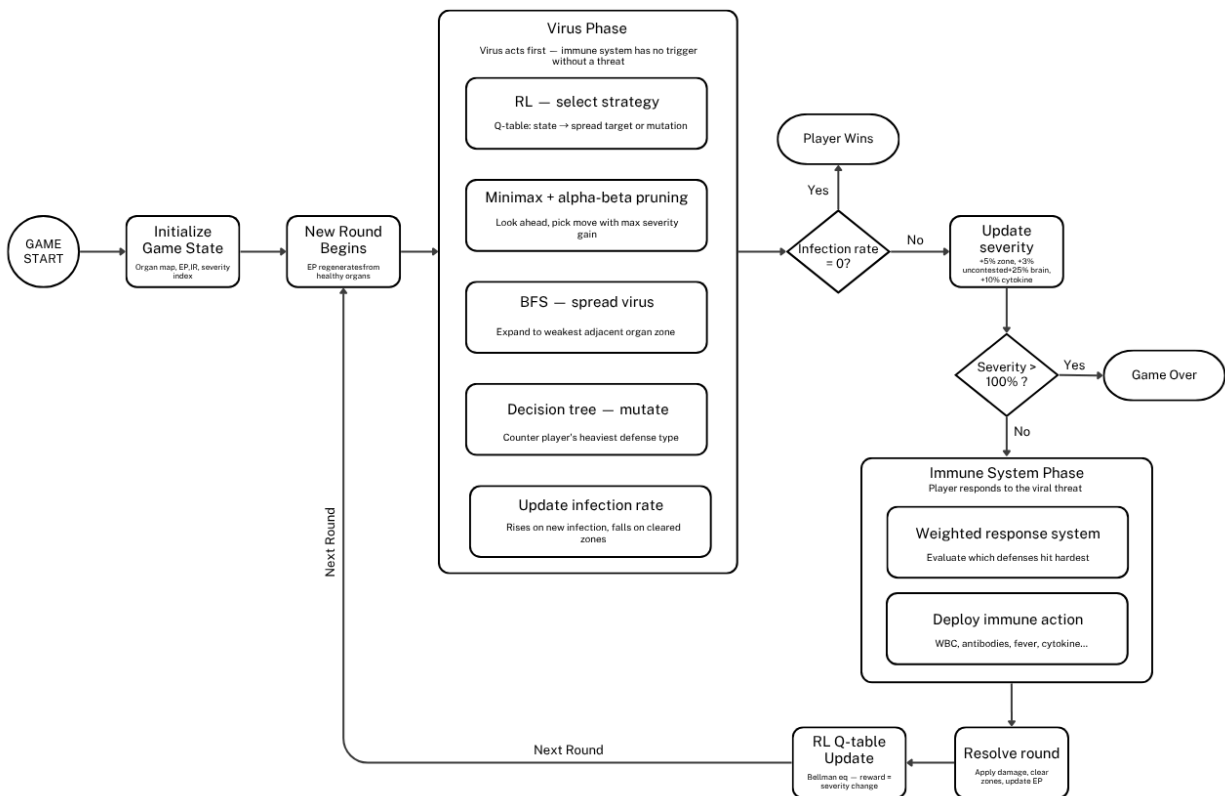


Figure 1. Software Architecture of Catch Me If You Can

Phase-by-Phase Explanation:

Phase 1: Game Start & Initialization

The game begins by initializing all core game state variables: the organ map (an undirected graph of six zones), the player's starting Energy Points (EP), the Infection Rate (set to a non-zero starting value representing the initial viral load in the Lungs), and the Severity Index (starting at 0%). Difficulty mode is also set here, which determines BFS spread resistance thresholds, Minimax tree depth, and mutation telegraph behavior.

Phase 2: New Round Begins — EP Regeneration

At the start of each round, the player's EP pool regenerates based on the number of currently healthy (uninfected) organ zones. Each healthy zone contributes a fixed EP amount passively. Infected zones contribute reduced or zero EP. This creates a compounding resource pressure: the more zones the virus controls, the fewer resources the player has to fight back with.

Phase 3: Virus Phase — RL Strategy Selection

The virus acts first every round. The RL module reads the current game state — which zones are infected, the player's remaining EP, and the current Severity Index — and queries the Q-table using an epsilon-greedy policy. If the exploration rate triggers, the virus selects a random action to discover new strategies. Otherwise, it selects the action with the highest expected Q-value, representing the move most likely to maximize the Severity Index gain based on past experience.

Phase 4: Virus Phase — Minimax Move Evaluation

Before committing to the RL-selected action, the virus runs Minimax with Alpha-Beta Pruning to evaluate the move's lookahead value. The algorithm projects the game state several turns forward, simulating both optimal virus actions and optimal player counter-responses at each branch. The virus selects the move with the highest guaranteed Severity Index gain — accounting for what the player is likely to do next. Tree depth scales with difficulty (Casual: 2, Epidemic: 4, Pandemic: 6), making the virus measurably smarter at higher settings without changing the algorithm.

Phase 5: Virus Phase — BFS Spread Execution

With the strategic target confirmed, BFS executes the actual spatial spread. Starting from all currently infected zones simultaneously, BFS expands outward to adjacent organ zones, sorted by ascending defense count. The virus infects the zone with the fewest active defenses first, increasing the Infection Rate and triggering a +5% Severity Index penalty for each newly occupied zone. In Pandemic mode, multiple independent BFS frontiers run concurrently from each starting entry point.

Phase 6: Virus Phase — Decision Tree Mutation

At the close of the virus phase, the Weighted Response System scores each of the player's currently active defenses by effectiveness. The defense type with the highest score is identified as the player's dominant strategy, and the Decision Tree traverses its hierarchy to select the appropriate counter-mutation. Compound mutations (available in Epidemic and Pandemic modes) can counter two defense types simultaneously. Mutations stack across rounds, progressively narrowing the player's effective options.

Phase 7: Infection Rate Check — Win Condition

After the virus phase resolves, the Infection Rate is evaluated. If it has been reduced to zero — meaning all active virus cells have been eliminated — the player wins immediately, regardless of the current Severity Index. This is an important design distinction: a host with high severity but no remaining virus is still a victory, because the threat has been completely eradicated. This check happens before the severity loss condition.

Phase 8: Severity Index Check — Loss Condition

If the Infection Rate is not zero, the Severity Index is updated and evaluated. It rises by +5% per newly infected zone, +3% per uncontested mutation each round, +10% if the Cytokine Burst was used, and +25% immediately if the Brain zone is infected. If the Severity Index reaches 100%, the host is considered beyond recovery and the game ends in a loss. The Severity Index can also decrease when zones are cleared and clean rounds are sustained.

Phase 9: Immune System Phase — Weighted Response System

The immune system phase begins with the Weighted Response System analyzing which of the player's deployed defenses performed most effectively during the previous rounds. This scoring is used both to inform the player's tactical feedback and to feed the virus's Decision Tree at the start of the next virus phase — closing the adaptive feedback loop that makes the AI feel genuinely reactive to player strategy.

Phase 10: Immune System Phase — Deploy Immune Action

The player selects and deploys one or more immune actions, each costing EP from the regenerated pool. Actions range from low-cost, broad-coverage options like White Blood Cells (10 EP) to high-cost emergency responses like the Cytokine Burst (50 EP, with a Severity Index penalty). Memory Cells allow the player to leverage immunological memory against previously seen mutations. The player must balance resource expenditure against the compounding threat of stacked mutations.

Phase 11: Resolve Round & RL Q-Table Update

All damage, zone clearances, and EP costs from both phases are applied simultaneously at the end of the round. Finally, the RL module updates the Q-table using the Bellman equation: the reward signal is calculated as the net change in Severity Index from the start of the round to its end. A reward spike (large severity gain) reinforces the action taken; a penalty (severity decrease due to player success) discourages it. The updated Q-table carries forward into the next round, making the virus incrementally smarter with every round it survives.

MEMBERS CONTRIBUTION

Members	Scores
LASAM, Vince Michael	5

MERCADO, Jeff Petterson	5
NICOLAS, John Rich	5
PAREDES, Lian Luigi	5